Relatório de Cobertura de Testes — ArtCraft

Aluno: Italo Dell Areti
Repositório analisado: github.com/storytold/artcraft (≈25,9 mil estrelas, ≈4,1 mil forks)
Commit analisado: `d4e4ac6` (11/10/2026)
Linguagem: Rust (workspace Cargo)
Ferramenta de cobertura: cargo-llvm-cov 0.9.1 (cobertura *source-based* do LLVM, `-C instrument-coverage`), rustc 1.94.1

1. O projeto

O ArtCraft é uma "IDE para artistas": um aplicativo desktop para criar imagens, vídeos e cenas 3D com modelos generativos de vários provedores (fal.ai, Midjourney, Grok, Sora, WorldLabs, Kinovi, Seedance, Kling, Veo etc.). O back-end em Rust é um *workspace* com 39 *crates*:

- **Clientes de API** (`crates/api_clients/*`): um cliente HTTP por provedor, mais o `artcraft_router`, que escolhe o provedor, estima o custo e despacha cada pedido de geração.
- **Esquemas** (`crates/schema/*`): *enums* e *tokens* compartilhados com o front-end e o banco SQLite de tarefas.
- **Bibliotecas utilitárias** (`crates/lib/*`): cookies, emulação de navegador, arquivos, MIME, URLs, JWT etc.
- **Aplicativos desktop**: `crates/desktop/artcraft` (Tauri) e `artcraft-native` (egui).

Os testes são majoritariamente **unitários e embutidos** no próprio arquivo de código (`#[cfg(test)] mod tests`), como é idiomático em Rust.

2. Execução dos testes

Ambiente: Linux x86-64, Rust estável 1.94.1, componente `llvm-tools-preview`.

```
export SQLX_OFFLINE=true CARGO_PROFILE_DEV_DEBUG=0
cargo llvm-cov --workspace --exclude artcraft --exclude tauri-plugin-http \
  --exclude artcraft_native --no-fail-fast --no-report
cargo llvm-cov report --html --output-dir out/artcraft/coverage
```

O script `scripts/artcraft-coverage.sh`, neste repositório, reproduz a execução completa. Três ajustes foram necessários:

1. **Crates excluídos.** Os três crates de interface gráfica (`artcraft`, `tauri-plugin-http` e `artcraft_native`) dependem de bibliotecas de GUI do sistema (webkit2gtk/GTK) e foram excluídos. Os 36 crates restantes contêm toda a lógica de negócio.
2. **`SQLX_OFFLINE=true`.** O crate `sqlite_tasks` usa as macros `sqlx::query!`, que validam o SQL em tempo de compilação. Seguindo o `_docs/dev_setup.md` do projeto, usou-se o cache offline `.sqlx/` em vez de um banco real.
3. **Headers para o bindgen.** A dependência `boring-sys2` (BoringSSL, usada pelo crate `wreq` para emular o TLS de navegadores) gera *bindings* com `bindgen`. A máquina não

tinha os headers internos do clang (`stddef.h`), então foram passados os headers equivalentes do GCC via `BINDGEN_EXTRA_CLANG_ARGS`.

Resultado	Quantidade
Binários de teste	39
Testes aprovados	5.675
Testes que falharam	**0**
Testes ignorados (`#[ignore]`)	716

Após a compilação, toda a suíte executou em menos de um minuto.

Testes ignorados. Os 716 testes ignorados são **testes de integração contra as APIs reais** dos provedores. Eles leem chaves e cookies de caminhos locais da máquina do desenvolvedor, por exemplo:

```
#[ignore] // requires real API key
async fn poll_completed_image_job() {
    let secret =
std::fs::read_to_string("/Users/bt/Artcraft/credentials/fal.api_key.txt")
```

Por isso, não podem rodar em CI nem nesta análise. Isso explica boa parte da baixa cobertura dos clientes HTTP (seção 6).

3. Resultados gerais

O LLVM mede **regiões** (trechos de código com o mesmo contador de execução, algo próximo de *statements*), **funções** e **linhas**. A cobertura de *branches* só existe no compilador *nightly* (`-Z coverage-options=branch`) e por isso não está disponível no Rust estável usado aqui.

Métrica	Cobertos / Total	Cobertura
Regions	109.629 / 164.612	**66,60%**
Functions	9.049 / 13.849	**65,34%**
Lines	70.417 / 111.858	**62,95%**

Distribuição por arquivo (linhas): dos 1.577 arquivos, 436 (28%) têm 100% de cobertura e 198 ficam entre 90% e 100%. Na outra ponta, 80 ficam entre 70% e 90%, 91 entre 50% e 70% e **772 (49%) abaixo de 50%**, dos quais 689 estão em 0%. A distribuição é **bimodal**: os arquivos ou são quase totalmente cobertos, ou não são cobertos.

Código de produção × código de teste

Como os testes ficam no mesmo arquivo do código, a métrica do LLVM soma as linhas dos dois. Separando as linhas pelo módulo `#[cfg(test)] mod ...` de cada arquivo (com base no LCOV gerado pela própria ferramenta), o resultado é:

Parte do código	Linhas cobertas / total	Cobertura
Código de produção	23.700 / 45.800	**51,7%**
Código dentro de `#[cfg(test)]`	45.786 / 60.959	75,1%

Ou seja, **57% das linhas medidas são código de teste**, e elas elevam a média geral. A cobertura real do código de produção é de cerca de **52%**, cerca de 11 pontos abaixo dos 63% do relatório. As linhas de teste não cobertas são justamente o corpo dos 716 testes ignorados.

4. Cobertura por crate

Na tabela, "Linhas" é o valor do relatório HTML e "Produção" desconta o código de teste:

Crate	Arquivos	Linhas (relatório)	Produção	Funções
`schema/public/enums`	113	98,4% (11.328/11.512)	96,7%	98,2%
`lib/data/primitives`	16	96,8%	94,5%	92,9%
`schema/public/tokens`	5	95,9%	96,4%	98,6%
`lib/url_utils`	4	88,0%	70,9%	96,2%
`lib/cookie_store_wrapper`	5	82,1%	71,1%	79,2%
`artcraft_router`	575	75,6% (33.178/43.895)	65,6%	77,5%
`grok_api_client`	31	69,9%	44,7%	72,6%
`gmicloud_client`	12	62,2%	39,6%	53,8%
`fal_client`	376	51,8% (13.181/25.440)	41,0%	56,4%
`kinovi_web_client`	48	45,7%	36,1%	53,0%
`midjourney_client`	27	28,7%	22,0%	35,5%
`artcraft_client`	105	28,6%	18,2%	17,5%
`worldlabs_consumer_client`	34	24,8%	16,3%	24,1%
`worldlabs_api_client`	22	24,4%	18,9%	38,3%
`grok_consumer_client`	68	20,2%	16,8%	21,9%
`openai_sora_client`	48	**13,0%**	13,0%	11,2%
`schema/database/sqlite_tasks`	15	**0,0%**	0,0%	0,0%
`lib/cookie_store`, `lib/caching/memory_store`, `lib/cloudflare_*`, `lib/data/web_base64`, `lib/build_metadata`	9	**0,0%**	0,0%	0,0%

5. Arquivos com mais e menos cobertura

Mais cobertos. São os arquivos de **dados e regras puras**:

- `enums/.../inference_model_type.rs` (936 linhas, 100%) e `enums/.../task_model_type.rs` (419 linhas, 100%). Os testes verificam a serialização (`serde`), `to_str` e `from_str` de cada variante, e usam `strum::EnumIter` para garantir que a lista `all_variants()` está completa.
- As **tabelas de custo** do roteador, como `providers/artcraft/seedance_2p0/cost.rs` (331 linhas, 100%, 30 testes), `seedance_2p0_fast/cost.rs` (273 linhas) e `tripo3d_h3p1/cost.rs` (219 linhas).

- `tokens/src/prefixes.rs` (197 linhas, 100%).

Mais linhas de produção não cobertas:

Arquivo	Linhas de produção não cobertas
`artcraft_router/.../generate_video/video_generation_request.rs`	306
`artcraft_router/.../generate_image/image_generation_request.rs`	204
`grok_consumer_client/.../grok_video_gen_chat_conversation.rs`	171
`kinovi_web_client/.../generate_video/generate_video.rs`	169
`worldlabs_consumer_client/.../upload_image_and_create_world_with_retry.rs`	159
`grok_consumer_client/.../upload_image_and_generate_video_with_retry.rs`	148
`kinovi_web_client/.../workflow_run_task/workflow_run_task.rs`	123
`artcraft_client/src/utils/login_challenge_client.rs`	109

6. O que não é coberto, e por quê

a) Chamadas HTTP aos provedores. Esta é a principal causa. As funções que montam a requisição, chamam `.send()` e interpretam a resposta só são exercitadas pelos testes `#[ignore]` que exigem credenciais reais. Em `worldlabs_api_client/.../generate_world.rs`, por exemplo, a função `generate_world` (linhas 33–92: envio, verificação de `status.is_success()` e tratamento de erro HTTP) tem 0% de cobertura, e as outras ~355 linhas do arquivo são 5 testes `#[ignore]`. Os clientes "consumer" (`grok_consumer_client`, `worldlabs_consumer_client`, `openai_sora_client`), que simulam o navegador contra sites web, têm entre 13% e 25%. Não existe nenhum teste com servidor HTTP falso (como `wiremock` ou `httpmock`) que permita testar esse código sem rede.

b) Despacho no roteador. No `artcraft_router`, os *enums* de requisição (`VideoGenerationRequest`, `ImageGenerationRequest`) têm um `match` gigante com uma variante por modelo. `estimate_cost` é parcialmente coberto, mas `get_provider` (linhas 220–290) e `send_request` (linhas 371 em diante, uma chamada de rede por variante) não são. Cada braço do `match` é uma região distinta, e por isso esses dois arquivos lideram as linhas não cobertas.

c) Crates sem nenhum teste. `sqlite_tasks`, com as consultas SQL da fila de tarefas do aplicativo, `cookie_store`, `memory_store`, `cloudflare_errors`, `cloudflare_mitigation` e `web_base64` não têm nenhum `#[test]` e aparecem com 0%. O caso de `sqlite_tasks` é o mais relevante: as consultas são validadas pelo `sqlx` em tempo de compilação, mas o comportamento (inserir, atualizar status e listar tarefas) não é testado.

d) Código legado. `fal_client/src/requests_old/` (150 arquivos, 7.428 linhas) contém a API antiga baseada em *webhooks*, com **5,9%** de cobertura, e ainda é referenciada pelo roteador. Sem esse diretório, o `fal_client` passaria de 51,8% para 70,8%.

7. Conclusão

O ArtCraft tem **cobertura moderada no agregado (63% das linhas)**, mas o número esconde dois extremos:

- **As regras de negócio puras são muito bem testadas.** *Enums* e *tokens* compartilhados, tabelas de custo e utilitários ficam acima de 95%, e nenhum dos 5.675 testes falhou.
- **A camada de integração com os provedores, que é o coração funcional do aplicativo, quase não é coberta**, porque os testes dependem de credenciais reais e ficam desabilitados. Descontando o próprio código de teste, a cobertura de produção fica em torno de **52%**.

Para melhorar, as recomendações são:

1. Introduzir testes com servidor HTTP falso (`wiremock`/`httpmock`) nos clientes de API, para exercitar o envio e o tratamento de respostas e erros sem rede nem credenciais.
2. Testar `sqlite_tasks` com um banco SQLite em memória (`sqlite::memory:`).
3. Remover ou isolar `requests_old`.
4. Excluir os módulos de teste da métrica, com o atributo `#[coverage(off)]` (*nightly*) ou com um filtro de linhas, para que o número reportado reflita o código de produção.

Anexo

O relatório HTML gerado pela ferramenta está em `artcraft-cobertura-html.zip`. Para navegar, abra `artcraft-coverage-html/index.html`.